

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1
по дисциплине «Кроссплатформенная разработка»
студента Абрамова Артёма Евгеньевича группы ПИН-252т

Пояснительная записка
Шифр работы От-43-ПИН-252т-1 ЛР
Специальность 09.03.04 Программная инженерия

Преподаватель

М.А. Мищенко

Студент

А.Е. Абрамов

Омск 2026

РЕФЕРАТ

Объектом выполненной работы является кроссплатформенное приложение «Каталог видов», разработанное на Kotlin с использованием Compose Multiplatform.

Целью данной работы является построение каркаса приложения: навигации между экранами, экрана-списка на моковых данных, переключения темы и локализации интерфейса, а также перехода по иерархии таксонов.

В работе применены архитектурный подход с разделением на слои domain, data и ui, однонаправленный поток данных (состояние экрана и намерения пользователя), библиотека Navigation 3 и механизм строковых ресурсов Compose Multiplatform.

Степень внедрения — для учебных целей.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 ПОСТАНОВКА ЗАДАЧИ	5
2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ.....	6
3 РЕАЛИЗАЦИЯ	7
3.1 Модели предметной области и репозиторий	7
3.2 Моковые данные	7
3.3 Состояние экрана и намерения	8
3.4 Навигация.....	9
3.5 Тема	11
3.6 Локализация.....	12
3.7 Поиск по названию таксона	13
4 ПРОВЕРКА РАБОТЫ.....	14
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	19

ВВЕДЕНИЕ

Кроссплатформенная разработка позволяет описать интерфейс и логику приложения один раз и собрать их под несколько платформ. Технология Compose Multiplatform расширяет декларативный подход Jetpack Compose на настольные системы и веб: общий код лежит в одном модуле, а платформенные точки входа только запускают его.

Предметной областью приложения выбран каталог биологических видов. Источник данных - открытый программный интерфейс GBIF Species API, который отдаёт сведения о таксонах: научное название, ранг, родительский таксон, число потомков. На первой вехе курса сеть не используется: приложение работает на моковых данных, повторяющих форму ответа сервера, поэтому на следующей вехе моки заменяются сетевым источником без изменения остального кода.

Цель работы - разработать каркас кроссплатформенного приложения, то есть такую основу, которая не рассыпается при обращении с ней: при переключении темы, смене языка, переходе на экран детали и возврате назад.

Для достижения цели поставлены следующие задачи:

- создать проект Kotlin Multiplatform из мастера JetBrains с таргетами Android, настольная система и веб;
- разложить код по слоям domain, data и ui так, чтобы зависимости сходились к слою предметной области;
- подготовить моковый набор данных не менее чем из двадцати таксонов по форме программного интерфейса;
- реализовать экран-список, экран детали таксона и экран дочерних таксонов, позволяющий ходить по дереву любой глубины;
- организовать навигацию на библиотеке Navigation 3, где маршрут является объектом, а не строкой;
- вынести цвета в тему и реализовать переключение светлого и тёмного оформления;
- вынести все подписи интерфейса в строковые ресурсы и добавить вторую локаль;
- реализовать поиск по названию таксона;
- проверить собираемость всех таргетов и включить отчёт компилятора Compose.

1 ПОСТАНОВКА ЗАДАЧИ

Приложение представляет собой каталог биологических видов и состоит из трёх экранов: списка таксонов, детали таксона и списка дочерних таксонов. Список показывает научное название таксона, его номер и бейдж ранга. Деталь содержит сведения, которых нет в списке: полное научное название, описание таксона и кнопку перехода к дочерним таксонам. С детали доступен переход к дочерним таксонам, а с них - к их потомкам, что позволяет ходить по дереву любой глубины.

Источником данных на последующих вехах выступает программный интерфейс GBIF Species API по адресу <https://api.gbif.org/v1/species/search>. Он не требует ключа, поддерживает постраничную выдачу параметрами limit и offset и содержит около сорока восьми миллионов записей. Ограничение источника - rate limit на быстрые запросы, поэтому данные для моков снимались с обычного эндпоинта без параметров и содержат первые двадцать записей ответа.

Требования к каркасу, проверяемые при сдаче работы, приведены в таблице 1.

Таблица 1 — Требования к каркасу приложения

Требование	Способ выполнения
Навигация на Navigation 3, маршрут - объект	Маршруты описаны sealed-интерфейсом, аргумент лежит полем объекта
Мультиплатформенная сборка навигации	Библиотека navigation3-ui от JetBrains и navigation3-runtime от Google
Возврат выглядит как возврат	Заданы все три спецификации переходов, включая обратный и предиктивный
Подписи в строковых ресурсах, две локали	Каталоги values и values-ru, паритет ключей проверяется скриптом
Тема из токенов оформления	Цвета объявлены только в файле темы, поверхности читают их из схемы
Экран рисуется из состояния	В функцию экрана передаются состояние и обработчик намерений
Поиск по названию таксона	Ввод подстроки сужает список, запрос сохраняется при возврате с детали
Отчёт компилятора включён	Экраны остаются перезапускаемыми и пропускаемыми

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Код приложения разделён на три слоя. Слой `domain` содержит модели предметной области и описание репозитория в виде интерфейса. Слой `data` содержит моковые данные и реализацию репозитория. Слой `ui` содержит экраны, переиспользуемые компоненты, тему, навигацию и модели представления. Все зависимости направлены к слою `domain`: слой `ui` пользуется его моделями, слой `data` преобразует полученные данные в эти же модели.

Такое разделение решает практическую задачу: на следующей вехе моковый репозиторий заменяется сетевым, и вся правка происходит внутри папки `data`, а экраны и модели предметной области остаются неизменными.

Структура каталогов общего модуля приведена в листинге 1.

<code>shared/src/commonMain/kotlin/ru/omgtu/abramov/</code>	
<code>App.kt</code>	корень, где собираются тема, репозиторий, навигатор и фабрики
<code>Screen.kt</code>	маршруты приложения
<code>domain/</code>	модели и интерфейс репозитория
<code>data/</code>	моки и реализация репозитория
<code>utils/</code>	форматирование и фильтрация
<code>ui/list/</code>	состояние, намерения и <code>ViewModel</code> списка
<code>ui/AppTheme.kt</code>	светлая и тёмная палитры
<code>ui/model/</code>	модели представления
<code>ui/navigation/</code>	навигатор и хост навигации
<code>ui/detail/</code>	состояние, намерения и <code>ViewModel</code> детали
<code>ui/screens/</code>	вёрстка экранов
<code>ui/components/</code>	переиспользуемые элементы интерфейса
<code>shared/src/commonMain/composeResources/</code>	
<code>values/</code>	подписи интерфейса на английском
<code>values-ru/</code>	подписи интерфейса на русском
<code>drawable/</code>	иконки

Листинг 1 - Структура общего модуля

Логика каждого экрана вынесена в отдельный класс `ViewModel`. Внутри него состояние хранится в изменяемом потоке, а наружу отдаётся поток только для чтения, поэтому изменить состояние извне невозможно. Действия пользователя приходят в единственный метод обработки намерений.

Зависимости уровня приложения — репозиторий, навигатор и фабрики `ViewModel` - создаются в корневой функции приложения и передаются вниз. Фабрики обязательны: в общем коде нет отражения, с помощью которого `Android` создаёт `ViewModel` без аргументов, в вебе такого механизма не существует.

3 РЕАЛИЗАЦИЯ

3.1 Модели предметной области и репозиторий

Модели повторяют форму ответа программного интерфейса, включая типы идентификаторов. Фрагмент моделей и описание репозитория приведены в листинге 2.

```
data class Taxon(  
    val key: Int,  
    val scientificName: String,  
    val canonicalName: String?,  
    val rank: Rank,  
    val parentKey: Int?,  
    val numDescendants: Int,  
    val kingdom: String?,  
    val phylum: String?,  
    val description: String?,  
)  
  
enum class Rank {  
    KINGDOM,  
    PHYLUM,  
    CLASS,  
    ORDER,  
    FAMILY,  
    GENUS,  
    SPECIES  
}  
  
interface TaxonRepository {  
    suspend fun getTaxons(): List<Taxon>  
    suspend fun getTaxon(key: Int): Taxon?  
    suspend fun getChildren(parentKey: Int): List<Taxon>  
}
```

Листинг 2 - Модель таксона и интерфейс репозитория

3.2 Моковые данные

Моки содержат двадцать таксонов: девять царств и одиннадцать типов. Записи снимались с программного интерфейса, поэтому названия, ранги и число потомков являются настоящими. Ранги подобраны разными: царство и тип. Фрагмент моков приведён в листинге 3.

```

val mockTaxons: List<Taxon> = listOf(
    Taxon(
        key = 0,
        scientificName = "incertae sedis",
        canonicalName = "incertae sedis",
        rank = Rank.KINGDOM,
        parentKey = null,
        numDescendants = 3613,
        kingdom = "incertae sedis",
        phylum = null,
        description = null,
    ),
    Taxon(
        key = 1,
        scientificName = "Animalia",
        canonicalName = "Animalia",
        rank = Rank.KINGDOM,
        parentKey = null,
        numDescendants = 2981931,
        kingdom = "Animalia",
        phylum = null,
        description = "Key to Chalcovietnamicus species groups and species 1.
" +
            "Embolus screw-like, without large embolic disc (ED) hidden
between " +
            "palpal bulb and cymbium; retromargin of chelicerae with one
large fissidentate " +
            "tooth of four cusps (Logunov 2020: figs 5-6).",
    ), ...

```

Листинг 3 - Фрагмент моковых данных

3.3 Состояние экрана и намерения

Экран не обрабатывает нажатия самостоятельно. Ему передаются состояние и функция обработки намерений, а решение принимает ViewModel. Такой порядок нужен не только для стройности кода: если передать в экран целиком ViewModel, компилятор Compose признает её нестабильным типом, начнёт сравнивать по ссылке и перестанет пропускать перерисовку экрана. Реализация приведена в листинге 4.

```

data class TaxonListState (
    val query: String = "",
    val items: List<TaxonCardUi> = emptyList(),
)

sealed interface TaxonListIntent {
    data class CardClicked(val key: Int) : TaxonListIntent
    data class QueryChanged(val query: String) : TaxonListIntent
}

class TaxonListViewModel(
    private val navigator: Navigator,
    private val repository: TaxonRepository = TaxonRepositoryImpl(),
    private val parentKey: Int? = null,
) : ViewModel() {

    private val _state = MutableStateFlow(TaxonListState())
    val state: StateFlow<TaxonListState> = _state.asStateFlow()

    init {
        load(null)
    }

    private var loadJob: Job? = null

    private fun load(filter: String?) {
        loadJob?.cancel()
        loadJob = viewModelScope.launch {
            val base = if (parentKey == null) {
                repository.getTaxons()
            } else {
                repository.getChildren(parentKey)
            }
            val filtered = base.filterByName(filter)
            _state.update { it.copy(items = filtered.toCardsUi()) }
        }
    }

    fun onIntent(intent: TaxonListIntent) {
        when (intent) {
            is TaxonListIntent.CardClicked ->
                navigator.addToBackStack(Screen.Detail(intent.key))

            is TaxonListIntent.QueryChanged -> {
                _state.update { it.copy(query = intent.query) }
                load(intent.query)
            }
        }
    }
}

```

Листинг 4 - Состояние, намерения и ViewModel экрана-списка

3.4 Навигация

В библиотеке Navigation 3 стека маршрутов как отдельной сущности нет: стек - это обычный список объектов. Маршруты описаны sealed-интерфейсом, а аргумент лежит полем объекта, поэтому опечатку в аргументе

ловит компилятор, а не выполнение программы. В маршрут кладётся только идентификатор, сами данные экран получает из репозитория.

```
sealed interface Screen {
    data object List : Screen
    data class Detail(val key: Int) : Screen
    data class Children(val parentKey: Int, val parentName: String) : Screen
}

class Navigator {
    private val _navStack: MutableStateFlow<List<Screen>> =
        MutableStateFlow(listOf(Screen.List))
    val navStack = _navStack.asStateFlow()

    fun addToBackStack(screen: Screen) {
        _navStack.update { it + screen }
    }

    fun back() {
        _navStack.update { currentStack ->
            if (currentStack.size > 1) currentStack.dropLast(1) else
currentStack
        }
    }
}
```

Листинг 5 - Маршруты и навигатор

Хост навигации описывает переходы и сопоставляет маршрутам экраны. Все три спецификации перехода заданы явно: прямая, обратная и предиктивная. Это не оформление, а условие работы - значения по умолчанию для непрямых переходов вне платформы Android являются заглушками и приводят к ошибке во время выполнения.

```
NavDisplay(
    backStack = backStack,
    modifier = modifier,
    onBack = navigator::back,
    transitionSpec = { slide(SlideDirection.Start) },
    popTransitionSpec = { slide(SlideDirection.End) },
    predictivePopTransitionSpec = { slide(SlideDirection.End) },

    entryProvider = entryProvider {
        entry<Screen.List> {
            val viewModel: TaxonListViewModel = viewModel(factory =
listViewModelFactory)
            val state by viewModel.state.collectAsStateWithLifecycle()
            TaxonListScreen(state = state, onIntent = viewModel::onIntent)
        }

        entry<Screen.Detail> { screen ->
            val viewModel: TaxonDetailViewModel = viewModel(
                key = "detail- $\{screen.key\}$ ",
                factory = detailViewModelFactory,
            )
        }
    }
}
```

```

        extras = TaxonDetailViewModelFactory.extrasFor(screen.key),
    )
    val state by viewModel.state.collectAsStateWithLifecycle()
    state?.let { TaxonDetailScreen(taxon = it, onIntent =
viewModel::onIntent) }
    }

    entry<Screen.Children> { screen ->
        val factory = remember(screen.parentKey) {
            childrenViewModelFactory(screen.parentKey)
        }
        val viewModel: TaxonListViewModel = viewModel(
            key = "children-{$screen.parentKey}",
            factory = factory,
        )
        val state by viewModel.state.collectAsStateWithLifecycle()
        TaxonListScreen(state = state, onIntent = viewModel::onIntent)
    }
}
}

```

Листинг 6 - Хост навигации и создание ViewModel детали

Классу ViewModel детали задан ключ, составленный из идентификатора таксона. Без ключа обращение к ViewModel возвращает один экземпляр на весь хост навигации, и открытая повторно деталь показала бы предыдущую запись.

3.5 Тема

Тема представляет собой набор токенов оформления, который провайдер кладёт в неявный контекст композиции. Чтение токена является чтением состояния, поэтому при смене темы перерисовываются именно те элементы, которые её прочитали. Цвет, записанный числом прямо в коде элемента, в переключении темы не участвует, так как мимо контекста подписка на изменение не возникает. По этой причине все цвета объявлены в одном файле.

```

private val LightColors = lightColorScheme(
    primary = Color(0xFF2E7D32),
    onPrimary = Color(0xFFFFFFFF),
    surface = Color(0xFFFFFBFE),
    onSurface = Color(0xFF1C1B1F),
    background = Color(0xFFFFFBFE),
    onBackground = Color(0xFF1C1B1F),
)

private val DarkColors = darkColorScheme(
    primary = Color(0xFF81C784),
    surface = Color(0xFF1C1B1F),
    background = Color(0xFF141218),
)

```

```

@Composable
fun AppTheme(darkTheme: Boolean, content: @Composable () -> Unit) {
    MaterialTheme(
        colorScheme = if (darkTheme) DarkColors else LightColors,
        content = content,
    )
}

```

Листинг 7 - Палитры и цвет статуса запуска

Исключением из правила является цвет ранга таксона: он несёт смысл данных, поэтому не меняется вместе с темой. Литералы этого цвета всё равно объявлены в файле темы, а наружу отдаётся функция, выбирающая цвет по рангу. Переключатель темы размещён в заголовке приложения и хранит своё значение в памяти сеанса: сохранение настроек на диск относится к третьей вехе курса.

3.6 Локализация

Все подписи интерфейса вынесены в строковые ресурсы общего модуля. По ним создаётся класс доступа, поэтому в коде стоит не строковое имя, а сгенерированный идентификатор, и опечатка не компилируется. Квалификатор локали разрешается во время выполнения: один и тот же идентификатор даёт разный текст.

```

<!-- values/strings.xml -->
<string name="app_title">Species Catalog</string>
<string name="section_children">Children</string>
<string name="action_back">Back</string>
<string name="action_toggle_theme">Toggle theme</string>

<!-- values-ru/strings.xml -->
<string name="app_title">Каталог видов</string>
<string name="section_children">Дочерние таксоны</string>
<string name="action_back">Назад</string>
<string name="action_toggle_theme">Сменить тему</string>

Text(stringResource(Res.string.section_children))

```

Листинг 8 - Строковые ресурсы и их использование

Паритет ключей между локалями проверяется сценарием `tools/check-strings.py`: ключ, забытый в одной локали, при переключении языка оставил бы на экране чужой язык, и глазами такая ошибка находится не всегда. Локализуется интерфейс, а не данные каталога: научные названия таксонов и описания остаются на языке источника. Исключение составляют подписи

рангов - это закрытый перечень из семи значений, он сопоставлен строковым ресурсам через словарь.

3.7 Поиск по названию таксона

Поиск реализован через расширение репозитория, а не через новый метод внутри интерфейса: репозиторий не знает про фильтрацию, а поиск можно тестировать отдельно. Ввод подстроки сужает список, регистронезависимо, по научному и каноническому названиям.

При возврате с экрана детали запрос сохраняется: состояние хранится в `ViewModel`, которая живёт, пока экран списка находится в стеке. При отсутствии совпадений экран показывает сообщение «Ничего не найдено». На экране дочерних таксонов поле поиска скрыто, поскольку фильтрация по родителю не требуется (листинг 9).

```
suspend fun TaxonRepository.getTaxons(filter: String?): List<Taxon> =
    getTaxons().filterByName(filter)

fun List<Taxon>.filterByName(filter: String?): List<Taxon> {
    val needle = filter?.trim().orEmpty()
    if (needle.isEmpty()) return this
    return filter { it.scientificName.contains(needle, ignoreCase = true) }
}
```

Листинг 9 - Расширение репозитория и функция фильтрации

4 ПРОВЕРКА РАБОТЫ

Приложение собирается и запускается на всех таргетах проекта. Команды запуска приведены в листинге 10.

```
./gradlew :desktopApp:run - запуск на десктоп
./gradlew :webApp:wasmJsBrowserDevelopmentRun - запуск на браузер
./gradlew :androidApp:assembleDebug - запуск на андроид
python3 tools/check-strings.py - проверка локализации
```

Листинг 9 — Команды сборки и проверки

Ведущим таргетом первой вехи выбран веб, зачётными таргетами курса - настольная система, веб и Android.

Порядок приёмки работы приведён в таблице 2.

Таблица 2 — Порядок проверки работы

Действие	Ожидаемый результат
Запустить приложение и прокрутить список	Двадцать карточек, у каждой название, номер и бейдж ранга
Открыть карточку и нажать возврат	Деталь выезжает справа, возврат уводит экран вправо, список на месте
На детали открыть дочерние таксоны	Открывается список детей, возврат ведёт на деталь, а не в список. У таксона без детей пустой список
Ввести подстроку в поиск	Список сужается; возврат с детали сохраняет запрос. При несуществующем запросе показано «Ничего не найдено»
Нажать переключатель темы в заголовке	Темнеют заголовок, фон и карточки, подписи остаются читаемыми
Выполнить сценарий проверки локалей	Расхождений ключей нет, выводится количество проверенных ключей
Переключить язык в заголовке	Интерфейс на втором языке, данные каталога остаются на языке источника

Отчёт компилятора Compose включён в сборке общего модуля и появляется в каталоге `shared/build/compose_compiler`. Однако составление отчета не происходит.

Внешний вид приложения приведён на рисунках 1–4

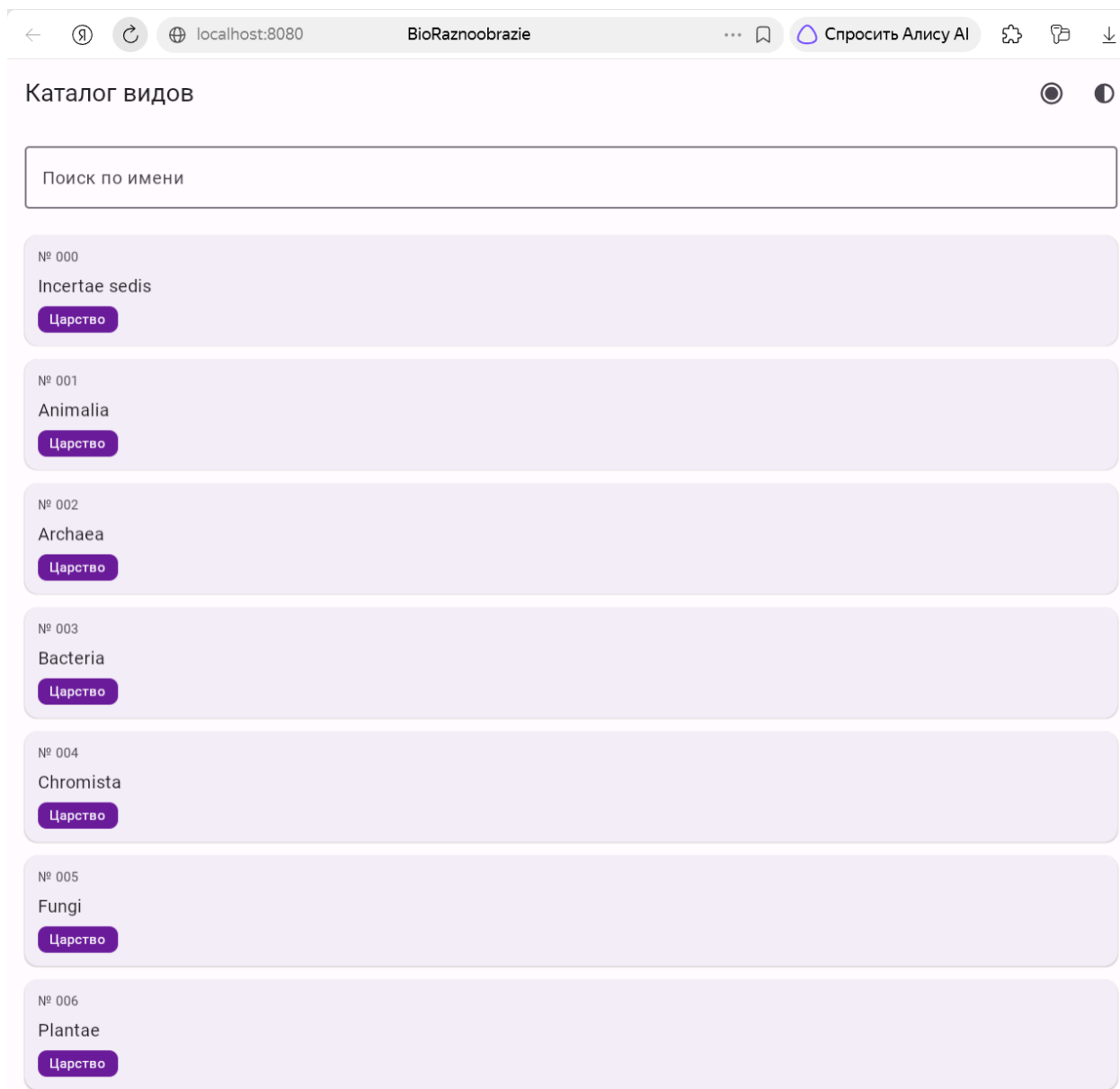


Рисунок 1 - Экран-список таксонов

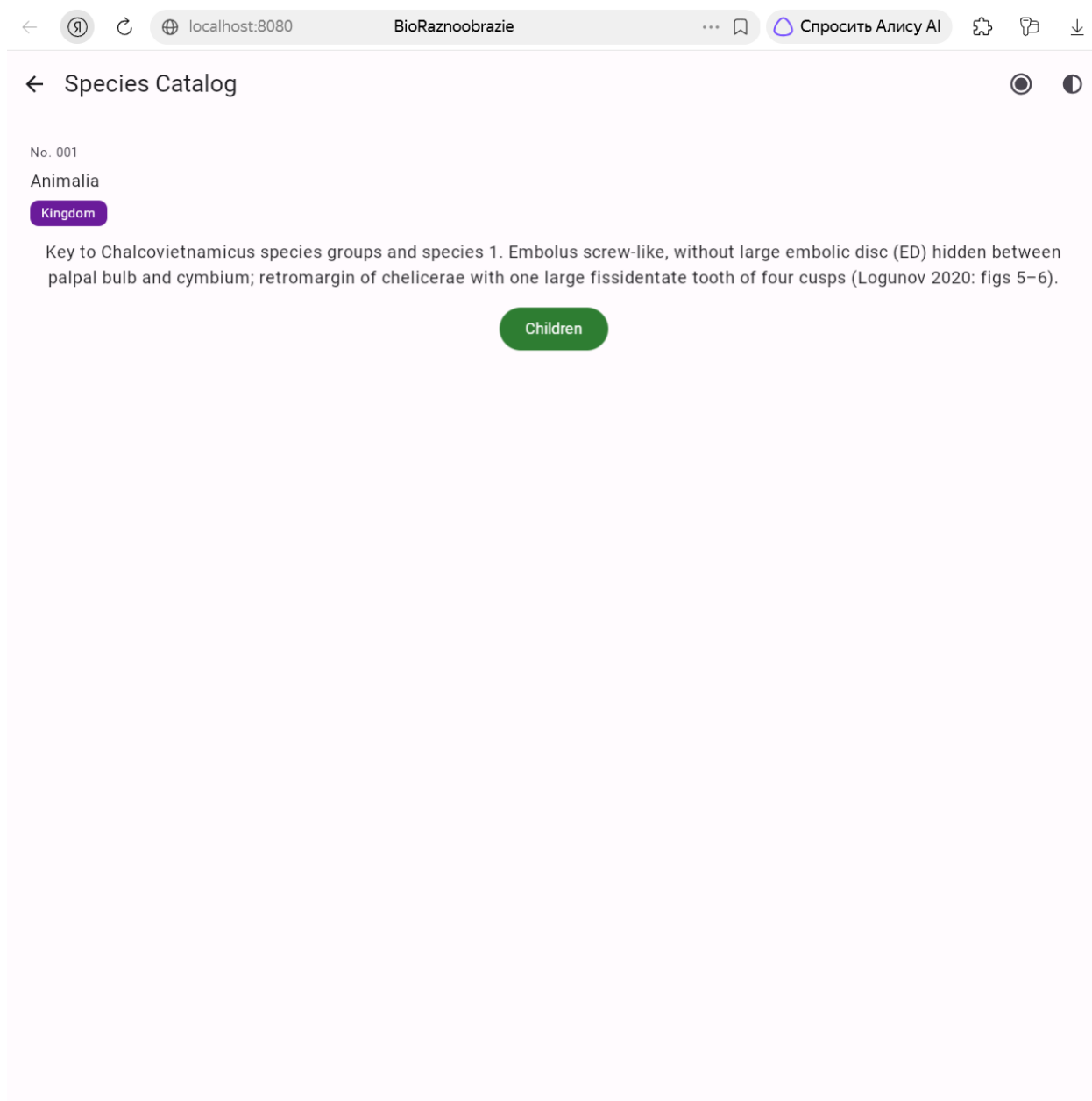


Рисунок 2 - Экран детали таксона с другим языком

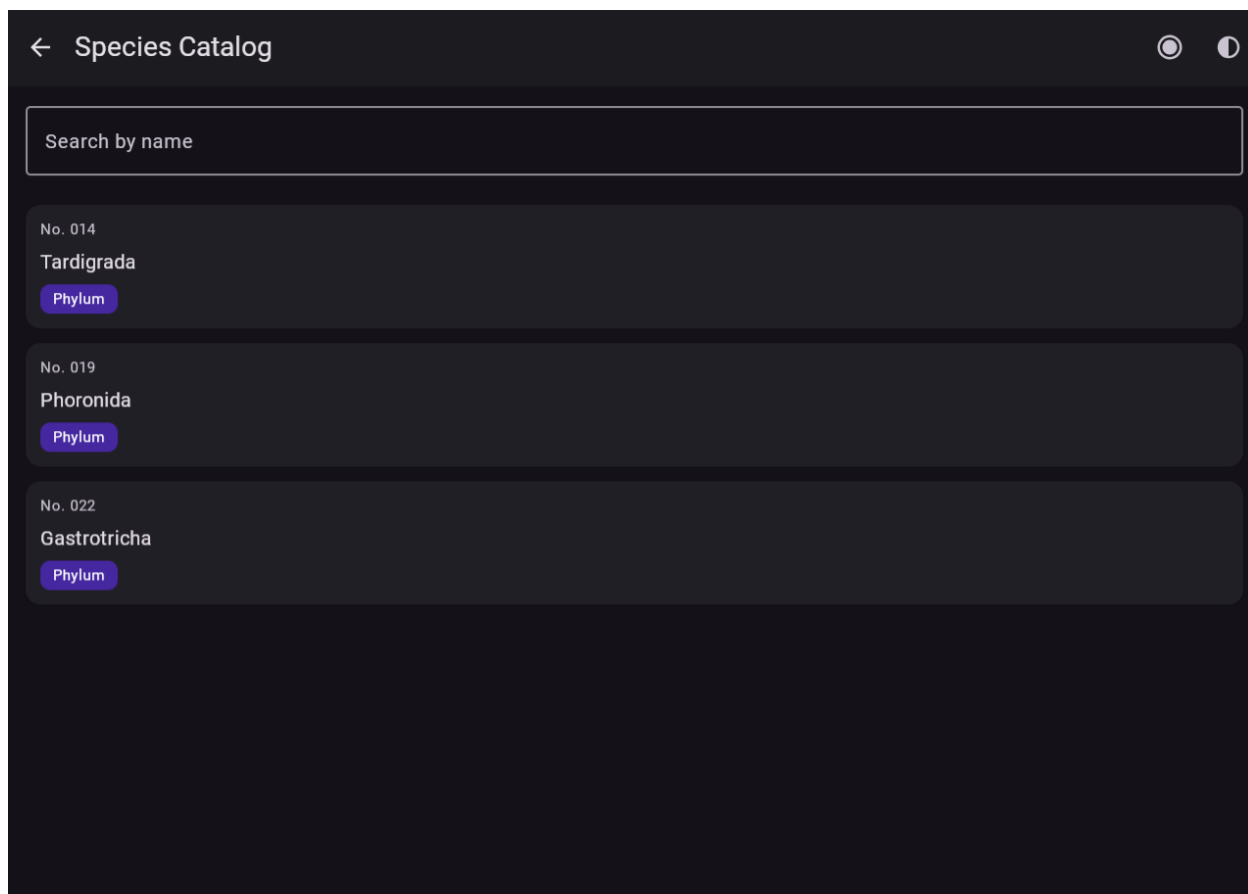


Рисунок 3 - Экран дочерних таксонов с тёмной темой

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы поставленная цель достигнута: разработан каркас кроссплатформенного приложения «Каталог видов» на Compose Multiplatform. Все поставленные задачи выполнены.

Код разложен по слоям domain, data и ui, зависимости направлены к слою предметной области, а моковый набор из двадцати таксонов повторяет форму программного интерфейса GBIF Species API. Благодаря этому замена моков на сетевой источник на следующей вехе затронет только слой data.

Реализованы три экрана и навигация между ними на библиотеке Navigation 3, где маршрут является объектом, а стек маршрутов — обычным списком. Экраны получают состояние и обработчик намерений, а не класс ViewModel целиком, поэтому все они остаются перезапускаемыми и пропускаемыми. Цвета вынесены в тему, переключение светлого и тёмного оформления работает, подписи интерфейса вынесены в строковые ресурсы и переведены на второй язык, а паритет ключей локалей проверяется отдельным сценарием. Поиск по названию таксона работает и сохраняет запрос при возврате с экрана детали.

Практическая ценность работы состоит в полученном опыте разделения приложения на слои и в понимании механизмов декларативного интерфейса: подписки на состояние при чтении, пропуска перерисовки и разрешения ресурсов по квалификатору локали. Полученный каркас служит основой для следующей вехи курса, на которой будут добавлены работа с сетью, страничная загрузка, состояния экрана и кэширование данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Compose Multiplatform: официальная документация [Электронный ресурс]. — URL: <https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html>
- 2) Navigation 3: официальная документация [Электронный ресурс]. — URL: <https://developer.android.com/guide/navigation/navigation-3>
- 3) Material Design 3: цветовые схемы и токены [Электронный ресурс]. — URL: <https://m3.material.io/styles/color/system>
- 4) Launch Library 2: описание программного интерфейса [Электронный ресурс]. — URL: <https://ll.thespacedevs.com/2.2.0/swagger/>
- 5) Compose compiler reports [Электронный ресурс]. — URL: <https://developer.android.com/develop/ui/compose/performance/stability/diagnose>